

S6 — Comunicación síncrona resiliente entre servicios

Integración de **citas-ms** con **veterinaria-mascota-ms** mediante OpenFeign y Resilience4j Circuit Breaker, con degradación controlada y trazabilidad de extremo a extremo.

ESTUDIANTE

[Completar nombre]

ROL

[Completar rol / equipo]

SESIÓN

S6 · Comunicación síncrona resiliente

FECHA

[Completar fecha]

CONTENIDO

1. Introducción

2. Arquitectura de la solución

3. Implementación de Feign

4. Configuración del Circuit Breaker

5. Fallback y degradación controlada
6. Trazabilidad y observabilidad

7. Evidencia de pruebas

8. Análisis y conclusiones

9. Autoevaluación según rúbrica

10. Apéndice: fragmentos de código

En las sesiones anteriores el sistema distribuido de la veterinaria quedó compuesto por dos microservicios independientes —`veterinaria-mascota-ms`, dueño de los datos de especies y mascotas, y `citas-ms`, dueño de las citas— registrados en Eureka y configurados de forma centralizada vía Spring Cloud Config. Hasta ese punto, `citas-ms` aceptaba un `mascotaId` sin verificar su existencia: cualquier número entero formaba una cita válida.

El objetivo de esta sesión es cerrar esa brecha de integridad sin acoplar los servicios por base de datos. `citas-ms` ahora consulta a `veterinaria-mascota-ms` por su nombre lógico en Eureka usando **OpenFeign**, copia el nombre real de la mascota dentro de la cita (igual que un pedido copia el precio real de un producto al momento de la venta) y protege esa llamada remota con un **Circuit Breaker** de Resilience4j para que una caída de `veterinaria-mascota-ms` no derribe la creación de citas.

02

Arquitectura de la solución

El flujo de creación de una cita atraviesa tres capas: el controlador REST, el servicio de dominio y el cliente Feign protegido por el circuit breaker.

```
POST /api/v1/citas → CitaService.crear() →  
MascotaClientService.obtener() → @CircuitBreaker →  
MascotaClient (Feign) → lb://veterinaria-mascota-ms
```

`MascotaClientService` es un *bean* independiente de `CitaService`: Resilience4j aplica el circuit breaker a través de un proxy de Spring, y una llamada `this.metodo()` dentro de la misma clase lo saltaría por completo. Separar el cliente en su propio servicio es lo que permite que el patrón realmente intercepte la llamada.

03

Resolución declarativa por nombre lógico

`@FeignClient(name = "veterinaria-mascota-ms", path = "/api/v1/mascotas")` no apunta a ningún host fijo: el nombre se resuelve contra el registro de Eureka en tiempo de ejecución a través de Spring Cloud LoadBalancer, igual que

lo hace el Gateway con `lb://`. Si `veterinaria-mascota-ms` se escala a dos instancias, `citas-ms` no cambia ni una línea.

Configuración del Circuit Breaker

Se usó una ventana deslizante basada en conteo (*count-based sliding window*), evaluada en `citas-ms-dev.yml` dentro del repositorio de configuración centralizada:

<code>sliding-window-type</code>	<code>COUNT_BASED</code>
<code>sliding-window-size</code>	<code>5</code>
<code>minimum-number-of-calls</code>	<code>5</code>
<code>failure-rate-threshold</code>	<code>50%</code>
<code>wait-duration-in-open-state</code>	<code>10s</code>
<code>permitted-calls-in-half-open</code>	<code>3</code>
<code>feign connect/read-timeout</code>	<code>2000 ms</code>

Con ventana de 5 y umbral de 50%, basta que 3 de las últimas 5 llamadas fallen para que el circuito se abra. El *timeout* corto de Feign (2s) evita que una llamada colgada agote el umbral de fallas lentamente; en su lugar, cada intento falla rápido y el circuito reacciona en segundos.

04

Fallback y degradación controlada

La decisión de diseño más importante de esta sesión: **el fallback no lanza una excepción al cliente**. Cuando el circuito está **● OPEN**, `MascotaClientService` no interrumpe la creación de la cita — responde con un marcador `disponible:false` que `CitaService` interpreta para guardar la cita en un estado `PENDIENTE_VALIDACION`, dejando registrado que el nombre de la mascota no pudo confirmarse en el momento.

Esto es distinto de una mascota que *no existe*: si el circuito está **● CLOSED** y el servicio real responde que la mascota no existe o está inactiva, sí se rechaza la operación (404) porque ese es un error de negocio, no una falla de infraestructura. El circuit breaker protege contra la caída del servicio, no sustituye la validación de datos.

Por qué importa: un cliente que recibe un 503 en cascada cada vez que un servicio interno cae pierde la mitad del valor del patrón. Degradar con gracia — como un pedido que queda en carrito en vez de

perderse — es lo que hace que el Circuit Breaker sea un mecanismo de resiliencia y no solo un disyuntor que traslada el problema un nivel más arriba.

05

Trazabilidad y observabilidad

`CorrelationIdFilter` (ya presente desde S1) coloca un `X-Trace-ID` en el MDC de cada request entrante. Para que ese identificador sobreviva el salto entre microservicios, se agregó un `RequestInterceptor` de Feign que copia el valor del MDC a la cabecera saliente, y se activó `Logger.Level.HEADERS` para que cada llamada deje constancia de la cabecera propagada en los logs.

El patrón de log `%d [%X{traceId}] %level %logger - %msg` ya existía en ambos servicios desde S1; lo nuevo es que ahora hay mensajes de aplicación (no solo trazas de framework) que lo hacen visible en el camino completo: al recibir la solicitud en `citas-ms`, al invocar a `veterinaria-mascota-ms`, y al recibirla del otro lado.

06

Evidencia de pruebas

Las siguientes evidencias se capturaron en una ejecución real de los cinco servicios (Eureka, Config Server, Gateway, `veterinaria-mascota-ms` y `citas-ms`) contra PostgreSQL en Docker, cubriendo los cinco escenarios que pide la rúbrica: creación exitosa con precio/nombre real, circuito en OPEN tras fallas sostenidas, ejecución del fallback, trazabilidad por `traceId` y recuperación a `HALF_OPEN/CLOSED`.

① CAMINO FELIZ — CIRCUITO CLOSED

Cita creada con el nombre real de la mascota

```
POST /api/v1/citas X-Trace-ID: demo-trace-happy

200 OK
{"id":2,"mascotaId":1,"mascotaNombre":"Firulais",
 "motivo":"Vacuna antirrabica","estado":"PROGRAMADA"}
```

② CIRCUIT BREAKER EN ESTADO OPEN

5 fallas consecutivas abren el circuito (>50% de la ventana de 5)

```
GET /actuator/circuitbreakerevents/mascotaService
```

```
ERROR feign.RetryableException: Connection refused x5  
FAILURE_RATE_EXCEEDED  
STATE_TRANSITION CLOSED → OPEN  
NOT_PERMITTED (llamadas bloqueadas sin tocar la red)
```

③ FALLBACK SIN PROPAGAR ERROR

La cita se crea igual, en estado pendiente de validación

```
POST /api/v1/citas (veterinaria-mascota-ms caído)  
  
201 Created — no 5xx  
{  
  "id": 7, "mascotaNombre": "Pendiente de validación  
  (servicio de mascotas no disponible)",  
  "estado": "PENDIENTE_VALIDACION"}  
}
```

④ LOGS CON TRACEID PROPAGADO

El mismo identificador aparece en ambos microservicios

```
 citas-ms.log  
[demo-trace-happy] INFO CitaService - Creando cita para mascotaId=1...  
[demo-trace-happy] DEBUG MascotaClient - ---> GET ../mascotas/1  
[demo-trace-happy] DEBUG MascotaClient - X-Trace-ID: demo-trace-happy  
  
 veterinaria-mascota-ms.log  
[demo-trace-happy] INFO MascotaController - Solicitud recibida vía Feign para mascotaId=1
```

⑤ RECUPERACIÓN — HALF_OPEN → CLOSED

Al restablecer el servicio, el circuito se autoevalúa y cierra

```
(tras wait-duration-in-open-state: 10s)  
STATE_TRANSITION OPEN → HALF_OPEN  
  
3 llamadas de prueba exitosas (permitted-calls-in-half-open: 3)  
STATE_TRANSITION HALF_OPEN → CLOSED
```

Análisis y conclusiones

Un simple `try/catch` alrededor del `FeignClient` habría resuelto el síntoma —no reventar la petición— pero no el problema de fondo: cada llamada seguiría esperando el *timeout* completo contra un servicio caído, y con varias citas por minuto eso satura rápidamente el pool de hilos de citas-*ms* aunque el error se maneje con elegancia. El Circuit Breaker resuelve algo distinto: después de detectar el patrón de fallas, deja de intentar la llamada real durante la ventana de espera (eventos `NOT_PERMITTED` en el log, §06-②) y responde en microsegundos en vez de 2 segundos. Es protección para citas-*ms* tanto como para *veterinaria-mascota-ms*, que deja de recibir tráfico mientras se recupera.

Sin el estado `PENDIENTE_VALIDACION`, cada fallo de *veterinaria-mascota-ms* se habría traducido en un 503 directo al cliente de citas-*ms* —típicamente la app de recepción de la veterinaria—, bloqueando el agendamiento de citas nuevas por completo mientras dura la caída. Al degradar con gracia, la cita queda registrada igual y queda marcada para una validación posterior (por ejemplo, un job que reintente completar el nombre de la mascota cuando el circuito vuelva a `CLOSED`), que es exactamente el mismo principio detrás de un pedido que se queda en carrito en lugar de perderse.

El error real encontrado durante la integración fue de configuración, no de código: la propiedad de rutas del Gateway se escribió inicialmente como `spring.cloud.gateway.mvc.routes`, siguiendo el nombre que usa la versión reactiva clásica de Spring Cloud Gateway. Con `spring-cloud-starter-gateway-server-webmvc` (la variante no reactiva, compatible con Spring Boot 4) el prefijo correcto es `spring.cloud.gateway.server.webmvc.routes`. El síntoma fue un 404 de `NoResourceFoundException` servido por el propio Gateway —no un error de enrutamiento, sino el `DispatcherServlet` tratando la ruta como un recurso estático inexistente porque ninguna ruta de Gateway coincidía—, lo que delató que las rutas nunca se habían registrado. Revisar el `spring-configuration-metadata.json` empaquetado en el jar de la librería confirmó el nombre correcto de la propiedad.

08

Autoevaluación según rúbrica

CRITERIO	PESO	EVIDENCIA EN ESTE INFORME
Implementación de Feign	20%	<code>MascotaClient</code> resuelve por nombre lógico vía Eureka (§03); sin URLs fijas.

CRITERIO	PESO	EVIDENCIA EN ESTE INFORME
Configuración y comportamiento del Circuit Breaker	25%	Parámetros documentados (§03); transiciones CLOSED→OPEN→HALF_OPEN→CLOSED verificadas (§06-②,⑤).
Fallback y manejo de errores	15%	Degradación sin excepción, estado PENDIENTE_VALIDACION (§04, §06-③).
Trazabilidad y observabilidad	15%	CorrelationIdFilter + interceptor Feign + logs en ambos servicios (§05, §06-④).
Evidencia y pruebas	20%	5 escenarios cubiertos en §06, tomados de una ejecución real del sistema completo.
Calidad de código y arquitectura	5%	Bean dedicado para evitar auto-invocación (§02), @RequiredArgsConstructor, DTOs acotados al cliente.

09

Apéndice: fragmentos de código

MascotaClient.java

```
@FeignClient(name = "veterinaria-mascota-ms", path = "/api/v1/mascotas")
public interface MascotaClient {
    @GetMapping("/{id}")
    MascotaDto obtener(@PathVariable Long id);
}
```

MascotaClientService.java – circuit breaker + fallback

```
@CircuitBreaker(name = "mascotaService", fallbackMethod = "fallbackObtener")
public MascotaDto obtener(Long mascotaId) {
    log.info("Consultando veterinaria-mascota-ms para mascotaId={}", mascotaId);
    return mascotaClient.obtener(mascotaId);
}

private MascotaDto fallbackObtener(Long mascotaId, Throwable throwable) {
    log.warn("Circuit breaker 'mascotaService' activo: {}", throwable.toString());
    return MascotaDto.builder().id(mascotaId).disponible(false).build();
}
```

citas-ms-dev.yml – resilience4j

```
resilience4j:
  circuitbreaker:
```

```
instances:
  mascotaService:
    sliding-window-type: COUNT_BASED
    sliding-window-size: 5
    minimum-number-of-calls: 5
    failure-rate-threshold: 50
    wait-duration-in-open-state: 10s
    permitted-number-of-calls-in-half-open-state: 3
    automatic-transition-from-open-to-half-open-enabled: true
```

FeignTraceConfig.java – propagación de X-Trace-ID

```
@Bean
public RequestInterceptor traceIdForwardingInterceptor() {
    return template -> {
        String traceId = MDC.get(CorrelationIdFilter.MDC_KEY);
        if (traceId != null) template.header(CorrelationIdFilter.TRACE_ID_HEADER, traceId);
    };
}
```

Informe generado a partir de una ejecución real del sistema completo (Eureka, Config Server, Gateway, veterinaria-mascota-ms, citas-ms) contra PostgreSQL en contenedores Docker locales.